

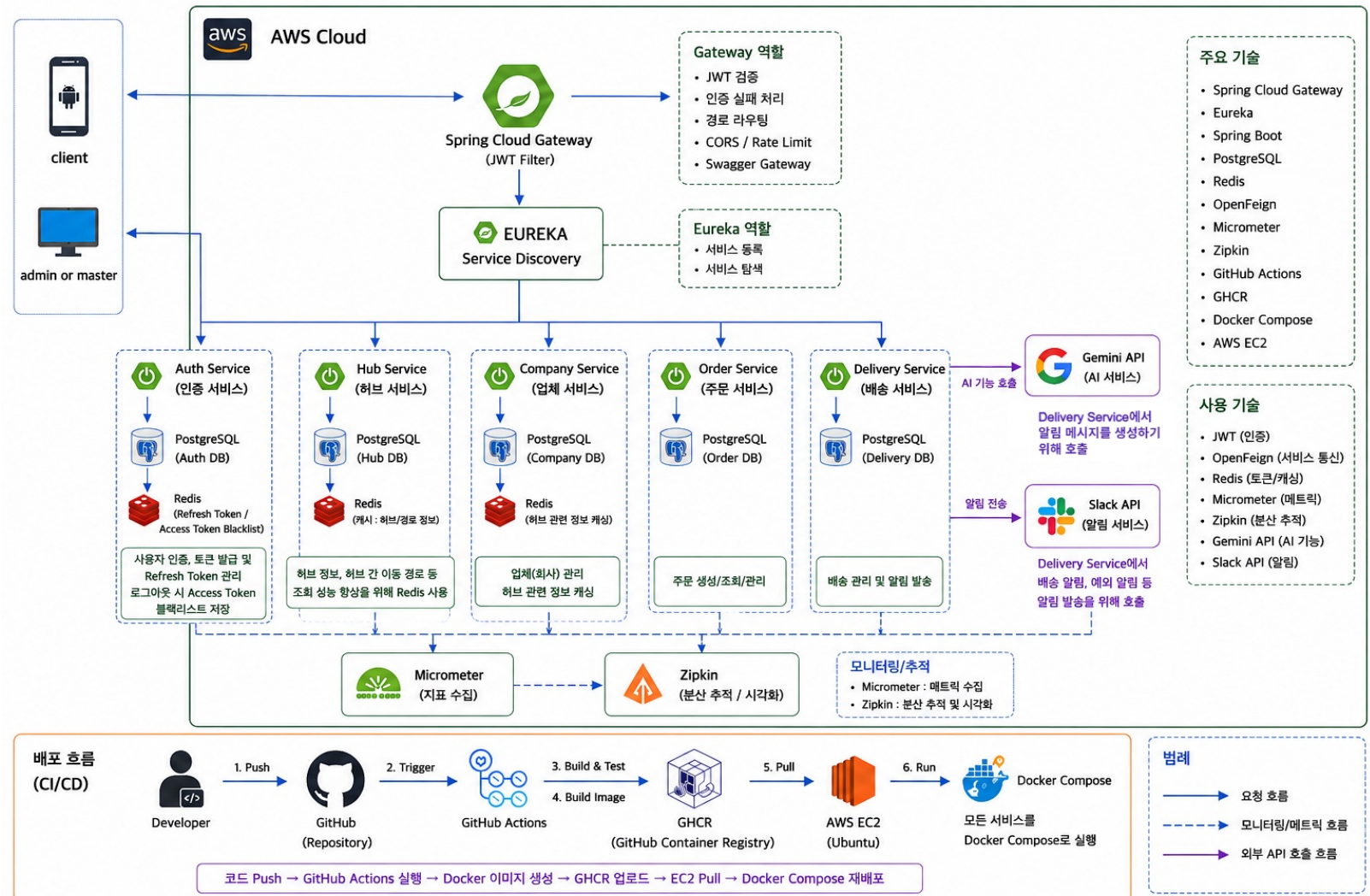
0404 — 주문부터 알림까지 연결한 MSA 물류 플랫폼

도메인별 책임을 분리하고, 인증 → 주문 → 배송 → 알림의 실패 지점을 기록·복구할 수 있도록 설계한 팀 프로젝트

0404

프로젝트 소개

Client 요청을 API Gateway에서 검증하고 각 도메인 서비스로 전달합니다. 서비스별 독립 DB와 Eureka 기반 탐색으로 주문과 배송, 외부 알림을 연결했습니다.



ERD와 API는 서비스의 책임 경계를 기준으로 설계

ERD는 서비스 내부 관계를, API는 서비스 간 계약과 데이터 흐름을 설명하도록 구성

p_company		
name	varchar(128)	필수
company_type	enum('DISTRIBUTOR', 'RECEIVER')	필수
hub_id	uuid	필수
address	varchar(255)	필수
created_at	timestamp	필수
updated_at	timestamp	필수
deleted_at	timestamp	선택
deleted_by	varchar(128)	선택

p_hub		
id	uuid	필수
name	varchar(128)	필수
address	varchar(255)	필수
website	varchar(255)	선택
created_at	timestamp	필수
updated_at	timestamp	필수
deleted_at	timestamp	선택
deleted_by	varchar(128)	선택

p_hub_routes		
id	uuid	필수
destination_hub_id	uuid	필수
distance	float	필수
created_at	timestamp	필수
updated_at	timestamp	필수
deleted_at	timestamp	선택
deleted_by	varchar(128)	선택

p_deliveries		
id	uuid	필수
order_id	uuid	필수
status	enum('PENDING', 'IN_PROGRESS', 'COMPLETED')	필수
origin_hub_id	uuid	필수
destination_hub_id	uuid	필수
destination_address	varchar(255)	필수
recipient_name	varchar(128)	필수
recipient_email	varchar(128)	필수
estimated_delivery_time	timestamp	필수
actual_delivery_time	timestamp	선택
created_at	timestamp	필수
updated_at	timestamp	필수
deleted_at	timestamp	선택
deleted_by	varchar(128)	선택

p_product		
id	uuid	필수
name	varchar(128)	필수
category	enum('FOOD', 'BEVERAGE', 'OTHER')	필수
sku	varchar(128)	필수
stock_status	enum('IN_STOCK', 'OUT_OF_STOCK')	필수
created_at	timestamp	필수
updated_at	timestamp	필수
deleted_at	timestamp	선택
deleted_by	varchar(128)	선택

p_users		
id	uuid	필수
username	varchar(128)	필수
password	varchar(128)	필수
email	varchar(128)	필수
role	enum('ADMIN', 'MANAGER', 'USER')	필수
created_at	timestamp	필수
updated_at	timestamp	필수
deleted_at	timestamp	선택
deleted_by	varchar(128)	선택

p_delivery_routes		
id	uuid	필수
origin_hub_id	uuid	필수
destination_hub_id	uuid	필수
distance	float	필수
created_at	timestamp	필수
updated_at	timestamp	필수
deleted_at	timestamp	선택
deleted_by	varchar(128)	선택

p_product_stock_movement		
id	uuid	필수
product_id	uuid	필수
quantity	int	필수
created_at	timestamp	필수

p_orders		
id	uuid	필수
order_number	varchar(128)	필수
customer_id	uuid	필수
delivery_id	uuid	필수
recipient_name	varchar(128)	필수
recipient_email	varchar(128)	필수
status	enum('PENDING', 'IN_PROGRESS', 'COMPLETED')	필수
created_at	timestamp	필수
updated_at	timestamp	필수
deleted_at	timestamp	선택
deleted_by	varchar(128)	선택

p_ai_messages		
id	uuid	필수
sender_id	uuid	필수
receiver_id	uuid	필수
message	text	필수
created_at	timestamp	필수
updated_at	timestamp	필수
deleted_at	timestamp	선택
deleted_by	varchar(128)	선택

p_stack_messages		
id	uuid	필수
message	text	필수
created_at	timestamp	필수
updated_at	timestamp	필수
deleted_at	timestamp	선택
deleted_by	varchar(128)	선택

p_delivery_managers		
id	uuid	필수
company_id	uuid	필수
manager_id	uuid	필수
status	enum('ACTIVE', 'INACTIVE')	필수
created_at	timestamp	필수
updated_at	timestamp	필수
deleted_at	timestamp	선택
deleted_by	varchar(128)	선택

p_order_items		
id	uuid	필수
order_id	uuid	필수
product_id	uuid	필수
quantity	int	필수
created_at	timestamp	필수
updated_at	timestamp	필수
deleted_at	timestamp	선택
deleted_by	varchar(128)	선택

p_delivery_manager_cursors		
id	uuid	필수
manager_id	uuid	필수
cursor_id	uuid	필수
created_at	timestamp	필수
updated_at	timestamp	필수
deleted_at	timestamp	선택
deleted_by	varchar(128)	선택

Common 모듈로 서비스 간 공통 계약 통일

실행 서비스가 아닌 공통 라이브러리 JAR로 응답·예외·인증·감사 규칙을 공유했습니다.

공통 영역	구현	왜 공통화했나?
API 응답·페이징	ApiResponse · PageResponse	서비스마다 다른 응답 구조와 페이징 형식으로 인한 계약 불일치를 줄이기 위해
예외 처리	ErrorCode · BusinessException	HTTP 상태·오류 코드·메시지를 같은 기준으로 반환하기 위해
인증 정보 전달	GatewayAuthenticationFilter · @CurrentUser	Gateway가 검증한 X-User-* 헤더를 Spring Security 인증 정보로 변환하기 위해
감사·논리 삭제	BaseEntity · JPA Auditing	생성·수정·삭제 정보는 JPA Auditing으로 자동기록, Soft Delete는 BaseEntity를 통해 모든 도메인에서 같은 방식으로 기록하기 위해
내부 서비스 호출	AuthHeaders · InternalCallInterceptor	InternalCallInterceptor가 Feign의 요청에 호출 서비스명을 자동으로 추가해, 수신 서비스가 허용된 내부 호출인지 구분하도록 하기 위해

 Common 변경이 모든 서비스에 영향을 줄 수 있기 때문에, 안정적인 공통 규칙만 포함하고 도메인 DTO와 비즈니스 로직은 각 서비스에 유지

JWT의 약점을 Redis 기반 RTR과 블랙리스트로 보완

➡ 1. 로그인 (Login)

사용자 → 인증 서버 → Access Token Refresh Token

🔄 2. 재발급 (Reissue)

기존 RT → Redis Lua Script (Atomic)
RT 검증 > 기존 RT 폐기 > 신규 RT 생성 → 새 토큰 발급

➡ 3. 로그아웃 (Logout)

로그아웃 요청 → Refresh Token 삭제 + AT jti 블랙리스트 등록

🛡️ API Gateway 검증 아키텍처

API Gateway 진입



JWT 서명 검증 & Redis 블랙리스트 검증



사용자 헤더 재생성 (UserId, Role)



Eureka 기반 서비스 전달

Refresh Token Rotation과 Access Token 블랙리스트를 Redis에서 원자적으로 관리하고, Gateway가 요청 진입 시 검증한다.

- Refresh Token Rotation으로 탈취된 토큰의 재사용 차단
- Access Token의 jti를 만료 시점까지 블랙리스트로 저장

- Lua Script로 검증과 교체를 원자적으로 처리, 동시 재발급 시 하나의 요청만 성공
- Redis 블랙리스트 조회 실패 시 Gateway가 503을 반환하는 **Fail-Closed** 정책 적용

Hub Service는 물류 거점과 이동 경로의 기준을 관리

✓ 허브·허브 경로를 한 서비스가 소유하고 조회 성능과 데이터 무결성을 함께 고려했습니다.

⚠ 기준 데이터: Hub · HubRoute

출발 Hub

물리 FK

HubRoute

활성 경로

도착 Hub

물리 FK

✕ 서비스 간에는 Hub UUID만 공유

🗄 조회·무결성 전략

READ OPTIMIZATION

JOIN FETCH + REDIS

(조회 최적화)

(캐시 전략)



캐시 TTL 10분

활성 경로 유일성

변경이 적은 허브 조회만 캐시하고 중복 경로는 DB 제약으로 방지

> HubRoute는 출발·도착 허브를 물리적 FK로 연결

> JOIN FETCH로 경로와 연관 허브의 N+1 방지

> 허브 단건 조회는 Redis TTL 10분 적용

> 활성 경로 중복을 유일 인덱스로 방지

Product·Company — 조회 성능과 재고 일관성을 함께 보호

읽기 성능은 Redis로, 재고 변경은 DB 잠금·먹등 기록·보상 트랜잭션으로 분리했습니다.

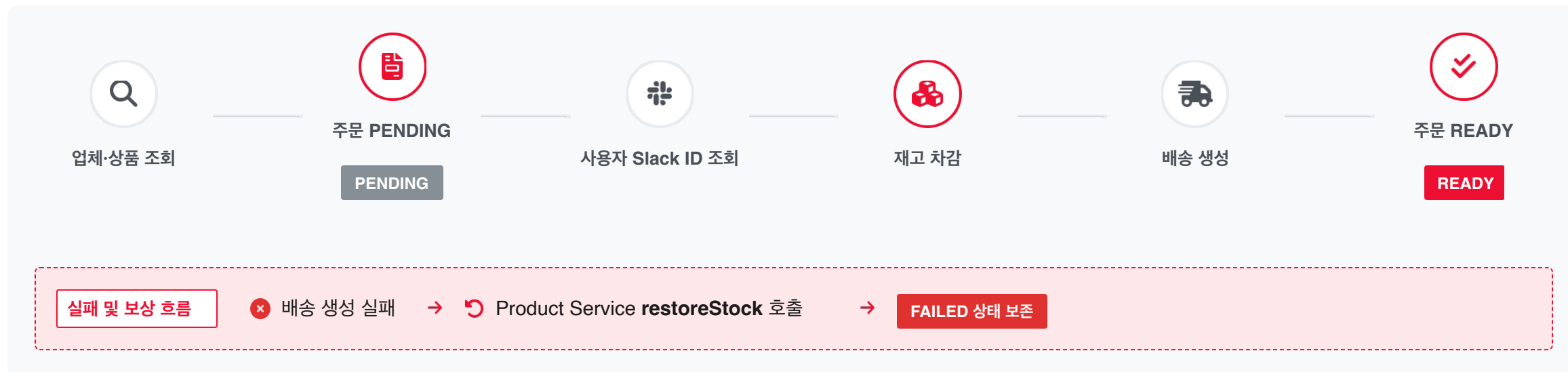
영역	발생한 문제	적용한 해결
Redis 캐싱	Hub·User 반복 조회로 DB 부하 증가	변경 빈도에 따라 Hub 10분 · User 2분 TTL 적용
동시성 제어	동시 재고 변경 시 Lost Update 가능	PostgreSQL 비관적 락(SELECT FOR UPDATE) · 타임아웃 3초
먹등성	네트워크 재시도 시 재고 중복 반영 가능	referenceld + DB UNIQUE · 락 획득 후 중복 검사
Saga 보상	독립 DB라 전체 흐름을 한 번에 롤백할 수 없음	후속 작업 실패 시 Product Service의 재고 복구 API 호출
현재 한계	보상 요청과 서비스 간 통신도 실패할 수 있음	재시도·모니터링·Kafka 기반 보상 이벤트로 확장 필요



주문 → 비관적 락 → referenceld 검증 → 재고 차감 → 배송 실패 시 재고 복구

Order — 주문 생성의 전체 흐름을 조정

Order Service는 Orchestration Saga로 정상 처리와 실패 보상을 조정하고 주문 상태를 명확히 남긴다.



 각 서비스가 독립된 DB를 사용해 주문·재고·배송을 하나의 트랜잭션으로 묶을 수 없어, 각 서비스의 작업을 단계적으로 호출하는 **Orchestration Saga** 적용

 재고 차감 후 실패하면 보상 트랜잭션(restoreStock) 수행으로 **정합성 유지**

 재고의 최종 검증과 변경 책임은 Product Service가 전담하여 **데이터 무결성 확보**

 주문 당시의 상품명과 가격은 Snapshot으로 저장하여 과거 이력의 **불변성 보장**

| Delivery — DB 기반 배정순서 공유로 담당자를 공정하게 배정

✓ 담당자 그룹별 마지막 배정 ID를 DB에 저장해 재시작과 다중 인스턴스 환경에서도 순번을 공유한다.

! 기존 문제: 메모리 기반 순번

Instance A

NEXT: 1

Instance B

NEXT: 5

Instance C

NEXT: 3

✕ 인스턴스 간 순번 불일치 및 재시작 시 초기화 발생

- > HUB 담당자는 전체 그룹에서 순환 배정
- > 마지막 ID 다음 담당자를 조회하는 **Round Robin** 방식

🗄️ 해결 방안: DB 기반 배정순서 공유

SHARED DATABASE CURSOR

LAST_ASSIGNED_ID: 1042



HUB 그룹 (전체)

COMPANY 그룹 (허브별)

모든 인스턴스가 단일 지점의 ID를 조회 및 갱신

- > COMPANY 담당자는 허브별 그룹에서 순환 배정
- > 주문 중복 방지와 취소 먹등성을 함께 검증

AI·Slack — 커밋 이후 비동기 처리로 배송 트랜잭션 보호

AFTER_COMMIT은 실행 시점을, @Async는 실행 스레드를 분리해 롤백 불일치와 응답 지연을 함께 해결합니다.

배송 저장 커밋



AFTER_COMMIT



@Async 리스너



Gemini ≤ 3회



Slack 알림

⚠️ 직접 호출의 문제

AI 응답 시간만큼 배송 생성 API도 함께 지연

배송 롤백 후에도 발송된 알림은 되돌릴 수 없음

비동기 예외는 호출자에게 전파되지 않아 추적이 어려움

🛡️ 적용한 설계

필요한 값만 담은 record 이벤트로 Lazy Loading 방지

PENDING 저장을 Writer 별도 빈의 REQUIRES_NEW로 독립 커밋

결과 반영도 별도 트랜잭션으로 처리해 실패 이력 보존

일시 오류만 최대 3회 재시도해 무한 재시도 방지

Gemini 30초 · Slack 연결 3초/응답 10초 Timeout

▶▶ 발견한 함정과 해결

CallerRunsPolicy는 포화 시 요청 스레드가 AI 호출을 대신 수행

AbortPolicy + 로깅으로 배송 응답에 지연이 전파되는 것을 차단

self-invocation을 피하도록 Writer를 별도 빈으로 분리

어려웠던 점은 “왜 이 전략인가”를 기준으로 해결

기술을 사용한 이유는 강의에서 다뤘기 때문이 아니라, 각 실패 지점의 책임과 복구 조건에 맞았기 때문입니다.

어려웠던 점	적용한 해결	왜 이 전략을 선택했나?
여러 서비스에 걸친 주문 실패	Saga + 보상 트랜잭션	단일 트랜잭션이 불가능해 완료된 작업을 역순으로 복구하기 위해
로그아웃 후에도 유효한 JWT	RTR + Access Token 블랙리스트	토큰의 무상태 장점은 유지하면서 탈취·로그아웃 토큰의 재사용을 차단하기 위해
다중 인스턴스 담당자 순번	DB 기반 배정순서 공유	서버 메모리가 아닌 공유 DB에 마지막 배정 상태를 남겨 재시작 후에도 순서를 유지하기 위해
외부 API 지연·실패	AFTER_COMMIT + 비동기 처리	알림 실패가 핵심 배송 트랜잭션의 성공 여부에 영향을 주지 않도록 하기 위해
반복 조회와 N+1	선택적 캐싱 + JOIN FETCH	정합성이 중요한 재고는 제외하고, 반복 조회 비용과 연관 데이터 추가 쿼리만 줄이기 위해

 실패 상태를 기록하고 복구 가능하게 만든 것이 기술 구현에서 가장 잘 된 점입니다.

잘 된 점, 협업 경험

기술 구현은 확인된 사실로 정리하고, 협업 문제와 느낀 점은 실제 팀 경험을 바탕으로 직접 작성합니다.

✔ 기술 구현에서 잘 된 점과 이유

핵심 작업은 동기, 알림은 비동기로 분리해 장애 영향 범위를 줄임

재고 복구와 FAILED 상태를 남겨 실패 원인과 처리 결과를 추적 가능하게 함

Gateway의 Role 검증과 서비스의 데이터 Scope 검증을 분리해 책임을 명확히 함

💬 협업 간 발생한 문제와 해결

Gateway의 Role 검증과 서비스의 데이터 Scope 검증을 분리해 책임을 명확히 함

데이터 구조나 정책이 바뀔 때마다 혼선이 있었으나, 지속적으로 공유하고, 공지 후에 작업함

개발 방식에 차이점이 있을 때 마다 논의를 통해 서비스 개발 방식을 확정함

👥 협업 방식에서 잘 된 점

한 사람의 작업이 다른 팀원의 작업에 영향을 줄 수 있는 부분의 공유가 잘 됨

PR시 2명 이상의 확인과 코드 리뷰를 통해 발생할 수 있는 문제를 사전에 방지함

편안한 분위기에서 자유롭게 의견을 제시할 수 있었음

🌱 팀 프로젝트를 진행하며 느낀 점

[박수연] 각 서비스의 연결점을 이해하고 해결하는데 많은 경험을 할 수 있었음

[백승환] MSA에서는 여러 서비스가 연결되어 있어, API 명세를 잘 작성해야 함을 느낌

[김준서] 다른 서비스와 같이 돌아가는 기능이 어떤 식으로 동작해야 할지 고민해볼 수 있었음

[이규민] 물류 시스템이 여러 서비스로 분리되어 있기 때문에, 서비스 간 협업의 중요성을 느낌

[한승욱] 공통 설정을 각 서비스에 일관되게 적용하고, 서비스 간 연동하는 법을 배웠음

개선과제

개선 과제는 현재 구현에서 확인된 동시성, 캐시 정합성, 서비스 장애 전파 문제를 기준으로 정리했습니다.

문제점	개선 적용 사항	기대 효과
동시 요청 시 재고 중복 차감 및 담당자 중복 배정 가능	Lock 또는 원자적 갱신 적용	데이터 정합성과 배정 정확성 확보
TTL 만료 전까지 변경 전 데이터가 조회될 수 있음	데이터 변경 이벤트 기반 캐시 삭제	변경 사항의 빠른 반영과 캐시 정합성 향상
Feign 호출 장애가 다른 서비스까지 전파될 수 있음	Circuit Breaker와 제한적 재시도 적용	일시 장애 복구 및 연쇄 장애 방지
분산 흐름과 동시 요청에 대한 검증 부족	주문 재고 배송 통합 및 동시성 테스트 강화	보상 트랜잭션과 동시성 제어의 신뢰성 확보
동기 호출 증가로 서비스 간 결합도와 응답 지연 증가	서비스 간 통신을 이벤트 기반으로 전환	서비스 결합도 감소 및 장애 격리